CS 335
Web Development
Spring 2025/2026

Milestone 4
Backend API and MongoDB Schema

TEAM Members

NAME: Shaikhah Muneer

ID: 2231172120

NAME: Hana Hato

ID: 2212172700

Project Title:

University Bus Management System

Date	Version	Description	Author/s
2026-05-03	4	Milestone 4 backend, MongoDB, authentication, driver assignment documentation	team

I. Backend API Development

1. Overview

The University Bus Management System backend is a RESTful API built with Node.js, Express, MongoDB Atlas, and Mongoose. The API supports user authentication, role-based access, bus and route management, trip assignment, booking, check-in/check-out, reports, subscriptions, demand requests, notifications, and bus status updates.

Main user roles:

- **Admin: manages users, buses, routes, and assigns trips to drivers.**
- **Driver: views assigned trips and updates trip status such as arrived, pickup complete, and completed.**
- **Student: views available trips, books seats, checks in/out, submits reports, and manages Beeyout subscription data.**

High-level request flow:

React Frontend → Axios request → Express Route → Controller → Mongoose Model → MongoDB Atlas

2. Authentication:

The API uses JWT-based authentication. During login or registration, the backend signs a token using JWT_SECRET from the .env file. The frontend stores the token and sends it in the Authorization header for protected requests.

Authorization: Bearer <JWT_TOKEN>

Authorization rules used in the project:

- **Public endpoints: register, login, and public trip listing can be accessed without a token.**
- **Protected endpoints: current user profile, driver actions, admin trip assignment, and management routes require a token.**
- **Admin-only action: creating/assigning a trip to a driver should require role admin.**
- **Driver/admin action: trip status updates should require role driver or admin.**

3. API Endpoints

Method	Endpoint	Description	Parameters / Body	Response	Sample Call
POST	/api/auth/register	Create a new user account	Body: fullName, email, phone, password, role, universityId, gender, city, block	201: token + user / 400: email exists or validation error	curl -X POST http://localhost:3001/api/auth/register -H "Content-Type: application/json" -d '{"fullName":"Hana","email":"hana@ku.edu.kw","password":"123456","role":"student"}'
POST	/api/auth/login	Login and receive JWT token	Body: email, password	200: token + user / 401: invalid credentials	curl -X POST http://localhost:3001/api/auth/login -H "Content-Type: application/json" -d '{"email":"student1@ku.edu.kw","password":"123456"}'
GET	/api/auth/me	Return currently logged-in user	Header: Authorization Bearer token	200: user / 401: invalid token	curl http://localhost:3001/api/auth/me -H "Authorization: Bearer TOKEN"
GET	/api/trips	Get all scheduled trips	Optional query: status, driver	200: list of trips	curl http://localhost:3001/api/trips
POST	/api/trips	Admin assigns/creates a trip for a driver	Header: admin token. Body: bus, driver, route, departureTime, arrivalTime, availableSeats	201: created trip / 403: not admin	curl -X POST http://localhost:3001/api/trips -H "Authorization: Bearer ADMIN_TOKEN" -H "Content-Type: application/json" -d '{"bus":"BUS_ID","driver":"DRIVER_ID","route":"ROUTE_ID","departureTime":"2026-05-04T07:30:00.000Z","arrivalTime":"2026-05-04T08:30:00.000Z","availableSeats":30}'
GET	/api/trips/:id	Get one trip by ID	Path param: id	200: trip / 404: not found	curl http://localhost:3001/api/trips/TRIP_ID
PATCH	/api/trips/:id	Update trip fields	Path param: id. Body: fields to update	200: updated trip / 400 validation error	curl -X PATCH http://localhost:3001/api/trips/TRIP_ID -H "Content-Type: application/json" -d '{"availableSeats":25}'
PATCH	/api/trips/:id/status	Update trip status	Body: status	200: updated trip / 400 invalid status	curl -X PATCH http://localhost:3001/api/trips/TRIP_ID/status -H "Content-Type: application/json" -d '{"status":"in-transit"}'
PATCH	/api/trips/:id/mark-arrived	Driver marks assigned Beeyout trip as arrived	Path param: id	200: tripStatus arrived + arrivedAt	curl -X PATCH http://localhost:3001/api/trips/TRIP_ID/mark-arrived
PATCH	/api/trips/:id/mark-pickup-complete	Driver marks pickup completed	Path param: id	200: tripStatus pickup-complete	curl -X PATCH http://localhost:3001/api/trips/TRIP_ID/mark-pickup-complete
PATCH	/api/trips/:id/complete	Driver completes trip	Path param: id	200: tripStatus completed	curl -X PATCH http://localhost:3001/api/trips/TRIP_ID/complete
GET	/api/bookings	Get all bookings	Admin/management use	200: list of bookings	curl http://localhost:3001/api/bookings
POST	/api/bookings	Student books a trip	Body: student, trip	201: booking created and availableSeats decreases / 400 no seats	curl -X POST http://localhost:3001/api/bookings -H "Content-Type: application/json" -d '{"student":"STUDENT_ID","trip":"TRIP_ID"}'
GET	/api/bookings/student/:studentId	Get bookings for one student	Path param: studentId	200: student booking list	curl http://localhost:3001/api/bookings/student/STUDENT_ID
PATCH	/api/bookings/:id/check-in	Check student into bus	Path param: booking id	200: checkInStatus checked-in, occupancy +1	curl -X PATCH http://localhost:3001/api/bookings/BOOKING_ID/check-in
PATCH	/api/bookings/:id/check-out	Check student out of bus	Path param: booking id	200: checkInStatus checked-out, occupancy -1	curl -X PATCH http://localhost:3001/api/bookings/BOOKING_ID/check-out

PATCH	/api/bookings/:id/cancel	Cancel booking	Path param: booking id	200: booking cancelled, availableSeats +1	curl -X PATCH http://localhost:3001/api/bookings/BOOKING_ID/cancel
GET/POST/PATCH/DELETE	/api/users	CRUD for users	Body depends on method	Standard CRUD response	curl http://localhost:3001/api/users
GET/POST/PATCH/DELETE	/api/buses	CRUD for buses	busNumber, capacity, busType, genderCategory, status	Standard CRUD response	curl http://localhost:3001/api/buses
GET/POST/PATCH/DELETE	/api/routes	CRUD for bus routes	routeName, routeType, departureLocation, destination, stops	Standard CRUD response	curl http://localhost:3001/api/routes
GET/POST/PATCH/DELETE	/api/reports	CRUD for student reports	student, route, trip, description, reportStatus	Standard CRUD response	curl http://localhost:3001/api/reports
GET/POST/PATCH/DELETE	/api/subscriptions	CRUD for Beeyout subscriptions	student, homeLocation, blockNumber, pickup times, payment info	Standard CRUD response	curl http://localhost:3001/api/subscriptions
GET/POST/PATCH/DELETE	/api/notifications	CRUD for notifications	sender, receiver, message, notificationType, readStatus	Standard CRUD response	curl http://localhost:3001/api/notifications
GET/POST/PATCH/DELETE	/api/demand-requests	CRUD for demand requests	student, trip, requestStatus	Standard CRUD response	curl http://localhost:3001/api/demand-requests
GET/POST/PATCH/DELETE	/api/bus-status-updates	CRUD for bus status updates	bus, driver, currentStatus, currentLocation	Standard CRUD response	curl http://localhost:3001/api/bus-status-updates

II. Revised Database Schema

Collection: users

Field	Type	Description
fullName	String, required	User full name
email	String, required, unique	Login email
phone	String	Phone number
password	String, required, select:false	Hashed password
role	Enum: student, driver, admin	Authorization role
universityId	String	University identifier
gender	Enum	User gender
city	String	Home city/area
block	String	Home block
isActive	Boolean	Account active status
timestamps	createdAt, updatedAt	Automatic dates

Collection: buses

Field	Type	Description
busNumber	String, required, unique	Bus plate/label
capacity	Number, required	Seat capacity
busType	Enum: beeyout, campus-off, college	Service type
genderCategory	Enum: female-only, mixed	Bus gender rule
status	Enum	Available/assigned/maintenance/inactive
isActive	Boolean	Whether bus can be used

Collection: routes

Field	Type	Description
routeName	String, required	Route display name
routeType	Enum	beeyout/campus-off/college
departureLocation	String	Start point
destination	String	End point
city	String	Area/city
blockStart, blockEnd	Number	Block range
stops	Array	Stop list with name/order/coordinates
isActive	Boolean	Route active status

Collection: trips

Field	Type	Description
bus	ObjectId ref Bus	Assigned bus
driver	ObjectId ref User	Assigned driver
route	ObjectId ref Route	Assigned route
departureTime	Date	Trip start
arrivalTime	Date	Trip end
tripStatus	Enum	scheduled/in-transit/arrived/pickup-complete/completed/cancelled
availableSeats	Number	Seats remaining
currentOccupancy	Number	Students currently checked in
arrivedAt, pickupCompletedAt, completedAt	Date	Driver action timestamps

Relationships: Uses ObjectId references to connect related collections.

Collection: bookings

Field	Type	Description
student	ObjectId ref User	Student who booked
trip	ObjectId ref Trip	Booked trip
bookingStatus	Enum	booked/cancelled/completed
checkInStatus	Enum	not-checked-in/checked-in/checked-out
checkInTime, checkOutTime	Date	Check in/out timestamps
qrCodeValue	String	QR/booking code

Index: compound unique index on student + trip to prevent duplicate booking for the same trip.

Relationships: Uses ObjectId references to connect related collections.

Collection: reports

Field	Type	Description
student	ObjectId ref User	Report owner
route	ObjectId ref Route	Related route
trip	ObjectId ref Trip	Related trip
description	String, required	Report text
reportStatus	Enum	open/in-review/resolved/rejected

Relationships: Uses ObjectId references to connect related collections.

Collection: demandrequests

Field	Type	Description
student	ObjectId ref User	Requester
trip	ObjectId ref Trip	Requested trip
requestStatus	Enum	pending/approved/rejected
requestTime	Date	Request date

Collection: subscriptions

Field	Type	Description
student	ObjectId ref User	Subscriber
homeLocation	String	Area/city
blockNumber	String	Home block
pickupTimeToUniversity	String	Morning pickup
pickupTimeToHome	String	Return pickup
serviceType	Enum: beeyout	Subscription service
paymentAmount	Number	Payment amount
paymentStatus	Enum	unpaid/paid/refunded
subscriptionStatus	Enum	active/paused/cancelled/expired

Relationships: Uses ObjectId references to connect related collections.

Collection: notifications

Field	Type	Description
sender	ObjectId ref User	Sender
receiver	ObjectId ref User	Receiver
message	String, required	Notification text
notificationType	Enum	arrival/booking/system/report/subscription
readStatus	Boolean	Read/unread

Relationships: Uses ObjectId references to connect related collections.

Collection: busstatusupdates

Field	Type	Description
bus	ObjectId ref Bus	Bus
driver	ObjectId ref User	Driver
currentStatus	Enum	available/in-transit/arrived/maintenance/inactive
currentLocation	String	Latest location/status note

Relationships: Uses ObjectId references to connect related collections.

III. Test cases

Test Case	Objective	Method	Input	Expected Output	Result
TC1 Register student	Verify new user can be created	POST /api/auth/register	New student data	201 success, user saved, token returned	Pass if user appears in MongoDB Atlas users collection
TC2 Login student	Verify JWT login works	POST /api/auth/login	student1@ku.edu.kw / 123456	200 success with token and user role student	Pass if token returned
TC3 Admin assign trip to driver	Verify admin can create assigned trip	POST /api/trips	bus id, driver id, route id, times, seats	201 created trip with driver field	Pass if trip appears in driver dashboard scheduled trips
TC4 Driver scheduled trips	Verify driver sees only assigned trips	GET /api/trips?driver=DRIVER_ID	Driver id	200 list of trips assigned to that driver	Pass if newly assigned admin trip appears
TC5 Driver mark arrived	Verify driver can update status	PATCH /api/trips/:id/mark-arrived	Trip id	200 tripStatus = arrived and arrivedAt is set	Pass if dashboard status changes
TC6 Student creates booking	Verify booking decreases available seats	POST /api/bookings	student id + trip id	201 booking created and availableSeats decreases by 1	Pass if booking exists and seat count updates
TC7 Student check-in	Verify bus occupancy increases	PATCH /api/bookings/:id/check-in	booking id	200 checkInStatus checked-in and currentOccupancy +1	Pass if progress/occupancy updates
TC8 Student check-out	Verify bus occupancy decreases	PATCH /api/bookings/:id/check-out	booking id	200 checkInStatus checked-out and currentOccupancy -1	Pass if occupancy decreases
TC9 Duplicate booking blocked	Prevent same student booking same trip twice	POST /api/bookings twice	same student id + same trip id	400 Student already booked this trip	Pass if second request fails

TC10 Invalid route returns 404	Verify API handles wrong URLs	GET /api/wrong-link	No body	404 Route not found	Pass if JSON error returned

IV. Deployment Guide and Installation Instructions

Local Installation

Project expected structure:

Milestone4/
  client/bus-app/
  server/

Backend setup:

cd server
npm install

Create server/.env:

PORT=3001
MONGO_URI=mongodb+srv://USERNAME:PASSWORD@cluster0.xxxxx.mongodb.net/university_bus_management?retryWrites=true&w=majority&appName=Cluster0
JWT_SECRET=university_bus_management_secret_key_2026
CLIENT_URL=http://localhost:3000

Seed and run backend:

npm run seed
npm run dev

Frontend setup in a second terminal:

cd client/bus-app
npm install
npm start

Test URLs:

http://localhost:3001/
http://localhost:3001/api/trips
http://localhost:3000

MongoDB Atlas Setup

- Create a free MongoDB Atlas account and cluster.
- Create a database user and password.
- In Network Access, allow your IP address or temporarily use 0.0.0.0/0 for development.
- Copy the Driver connection string.
- Replace <password> with your database user password.
- Add the database name after .net/: university_bus_management.
- Place the final URI in server/.env as MONGO_URI.
- Run npm run seed to create the mock data in Atlas.

GitHub Submission Steps

- Create a new repository on GitHub named university-bus-management-system or milestone4-university-bus.
- Do not upload node_modules, .env, or secret passwords.
- Add .gitignore before pushing.
- Commit the frontend folder, backend folder, README.md, and Milestone 4 documentation.
- Push to GitHub and submit the repository link.

Recommended .gitignore:

```
node_modules/
.env
.DS_Store
build/
dist/
*.log
```

Command-line steps:

```
cd Milestone4
git init
git add .
git commit -m "Milestone 4 backend and MongoDB implementation"
git branch -M main
git remote add origin
https://github.com/YOUR_USERNAME/YOUR_REPOSITORY_NAME
.git
git push -u origin main
```

If Git asks for identity:

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

IV. Contributions

Team member	Contribution	
	Who did what	Percentage (%)